

Autonomous Drone Precision Landing & PID Tuning Simulation

- AeroHaul 2026

Akansha Das

Anshum Bhargav

1. Overview

This project presents a Software-in-the-Loop (SITL) automation pipeline for multicopter drones using PX4 Autopilot, MAVSDK-Python and Gazebo. The system enables real-time telemetry, parameter tuning and autonomous mission execution over a local UDP connection. The objective was to autonomously navigate the drone and perform a precision landing on a designated helipad.

2. Problem Statement

The provided PX4 SITL environment featured an intentionally unstable drone that exhibited high-frequency oscillations and significant altitude drift, making autonomous flight impossible without first tuning the control system. The project was completed in two phases:

- **Attitude and Rate Stabilization:** The default PID parameters caused severe oscillations and unstable take-offs. The control loops were systematically tuned to achieve stable and predictable flight.
- **Autonomous Navigation & Target Landing:** Once stabilized, the drone autonomously navigated to the target location and executed a controlled precision landing on the designated helipad.

2. PID Tuning Process & Parameters Modified

Tuning Workflow and Control Loop Dynamics

- **Parameter Injection and Settle Heuristics:** The PID parameters were uploaded sequentially to the flight controller. After the upload process, a 2-second delay was introduced before any further operations. This allowed the PX4 parameter subsystem to complete all internal write operations and ensured that the updated values were fully applied before telemetry checks began.

- **Estimator Health Gating:** To ensure safe operation, the tuning workflow included a health verification stage based on PX4's EKF2 (Extended Kalman Filter) pre-arm checks. The script proceeded only after confirming that both the global position estimate and the home position were valid.
- **Rate Controller Optimization:** The roll and pitch rate controllers were tuned to reduce the aggressive over-correction observed during flight. Lower proportional gains helped suppress high-frequency oscillations, while matching integral gains across both axes produced a more balanced and predictable response.
- **Vertical Velocity Stabilization:** The altitude controller was refined by adjusting the vertical acceleration and velocity control parameters. These changes minimized altitude overshoot during autonomous take-off and reduced altitude sagging while the drone was in forward flight.

Modified Parameters

Parameter Group	Parameter Name	Final Tuned Value	Justification
Roll Rate	MC_ROLLRATE_P	0.16	Reduced to limit over-correction and suppress oscillations.
	MC_ROLLRATE_I	0.2	Increased to reduce steady-state error and improve tracking.
	MC_ROLLRATE_D	0.03	Maintained for stable rate response.
Pitch Rate	MC_PITCHRATE_P	0.16	Matched to roll controller to achieve a consistent response across both axes.
	MC_PITCHRATE_I	0.2	Improved steady-state tracking performance.
	MC_PITCHRATE_D	0.03	Provided damping similar to the roll controller.
Yaw Rate	MC_YAWRATE_P	0.20	Improved heading responsiveness during directional changes.
	MC_YAWRATE_I	0.10	Enhanced heading stability by reducing accumulated error.
	MC_YAWRATE_D	0.000	Left unchanged.
Altitude/ Z-Velocity	MPC_Z_VEL_P_ACC	3.70	Improved vertical response by reducing overshoot during take-off and descent.

	MPC_Z_VEL_I_ACC	2.00	Improved altitude regulation by reducing steady-state error.
	MPC_Z_VEL_D_ACC	0.000	Left unchanged.

3. Final Autonomous Flight Approach

To accomplish a precision touchdown on the designated target helipad, the system required precise coordinate translation and an automated navigation strategy.

Spatial Coordinate Projection

The target location was provided as local linear offsets from the drone's home position. Since PX4 navigation operates using global GPS coordinates, these local offsets were converted into latitude and longitude using a flat-Earth equirectangular approximation. The conversion accounts for meridian convergence by incorporating the cosine of the local latitude and providing sufficient accuracy for meter-scale navigation.

Two distinct operational architectures were evaluated for executing the flight plan:

Strategy A: PX4 Mission-Based Navigation (mission.py)

In this approach, the complete mission was generated within the script and uploaded directly to PX4, allowing the onboard mission controller to manage the flight.

- **Mission Execution:** The flight path was packaged as a mission plan containing the required waypoint and landing command. Once the mission was uploaded and started, PX4 assumed full control of waypoint navigation and mission execution.
- **Waypoint Behaviour:** The waypoint was configured with a stop-and-hold condition (`is_fly_through = False`), ensuring that the drone slowed down and established a stable hover before proceeding to the landing phase.
- **Landing Sequence:** After reaching the target waypoint, PX4 automatically transitioned into its native landing mode and completed the descent without requiring additional intervention from the external script.
- **Flight Monitoring:** To accurately determine mission completion, the implementation used a "was airborne" latch mechanism. This prevented the drone from being incorrectly identified as landed immediately after arming and ensured that landing was only confirmed after the vehicle had completed a full flight.

Strategy B: Manually Sequenced Guided Flight (landing_script_v2.py) (Selected Approach)

Unlike the mission-based approach, this method retained control within the external script. The flight was executed as a sequence of predefined phases, with each stage completed before moving to the next.

- **Altitude Management:** Guided reposition commands in PX4 require target altitudes to be specified as AMSL (Above Mean Sea Level). To generate the correct target altitude, the script continuously referenced the home position altitude and added the desired relative cruising height before issuing navigation commands.
- **Navigation Monitoring:** Instead of relying on the onboard mission controller, the script continuously monitored the drone's position throughout the flight. The Haversine formula was used to calculate the great-circle distance between the live GPS coordinates and the target helipad, allowing the script to determine when the drone had reached the desired location.
- **Landing Execution:** Once the drone was within the specified arrival tolerance (≤ 0.5 m), the navigation phase was terminated and a landing command was issued. The vehicle then performed a controlled vertical descent to complete the precision landing.

4. Challenges Faced & Mitigations

Several challenges were encountered during PID tuning and autonomous landing implementation, which were resolved through iterative testing.

- **Manual PID Tuning:** Initial tuning by modifying one parameter at a time was slow due to repeated simulation restarts. To improve efficiency, `pid_entry.py` automated parameter injection using asynchronous status monitoring and EKF2 health checks.
- **MAVSDK Telemetry Attribute Mismatch:** The navigation script failed because incorrect GPS telemetry attributes were used. Replacing them with `latitude_deg` and `longitude_deg` resolved the runtime errors.
- **Coordinate Frame Mismatch:** A mismatch between Gazebo's ENU and PX4's NED coordinate frames caused a consistent landing error of about 2.8 m. Remapping the target coordinates and using local NED Offboard position control corrected the issue.
- **Premature Landing Initiation:** Landing based only on horizontal distance caused drift during descent. Adding a horizontal velocity check before initiating landing resulted in more accurate touchdowns.

5. Conclusion

This project successfully implemented a Software-in-the-Loop (SITL) pipeline for autonomous multicopter flight using PX4 Autopilot, MAVSDK-Python, and Gazebo. Through PID tuning and autonomous navigation, the drone achieved stable flight and successfully completed a precision landing on the designated helipad, demonstrating a reliable approach for autonomous flight in a simulated environment.